```
  for index in {(|pairs|-1)/D .. 0} do
                          #1
    pushDown(pairs, index)
                 #2
```

#1 We start at the first internal node of the heap (`D` is the branching factor) and iterate till the root
#2 Ensure the sub-heap rooted at index is a valid heap

In total, in a binary heap, we will call `pushDown` `n/2` times, with an initial estimate of the upper bound of `O(n*log(n))`.
However, notice that the sub-heaps that only have leaves as children, have a height equal to 1, so they only need at most 1 swap to be fixed. And there is `n/2` of them. Likewise, sub-heaps with height 2, only need at most 2 swaps, and we hav> `n/4` of them.
Going up towards the root, the number of swaps increases but the number of calls to `pushDown` decreases at the same rate: finally we'll have only 2 calls for `pushDown` that requires at most `log2(n)-1` swaps, and only one call, the one on the root, that requires `log2(n)` swaps in the worst case.
Putting all these together, we see that the total number of swaps is given by:

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot O(h) = O\left(n \sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{h}{2^{h}} \right\rceil \right)$$

Since the last summation is limited by geometric series with seed 2, we have: